



COMPUTER SCIENCE

CS-305L: Parallel & Distributed Computing Lab

SUBMITTED BY

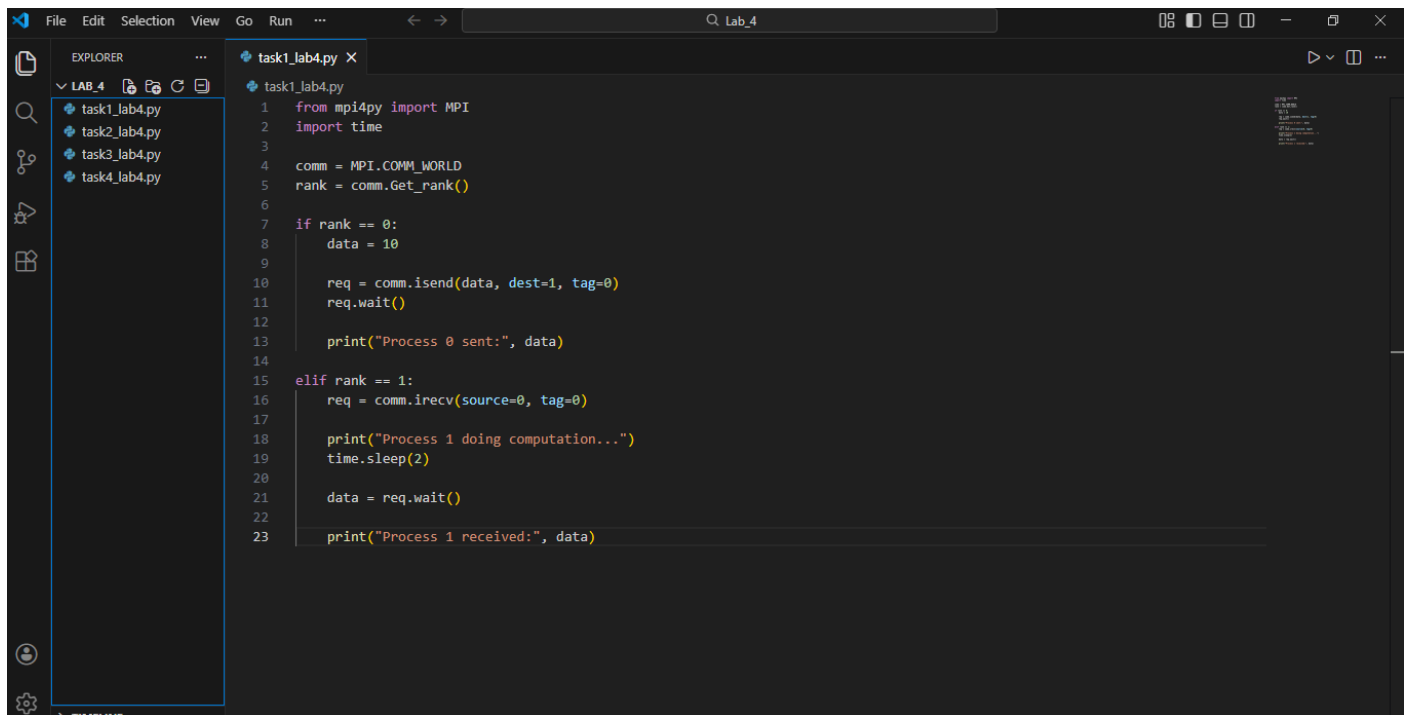
NAME : RIFAQ AJMAL
REG NO : 23MDBCS 435
SECTION : CS-A
SEMESTER : 6th

LAB 4

Task 1: Non-blocking Communication (Isend/Irecv)

- Implement an MPI program where **Process 0** sends an integer to **Process 1** using **nonblocking communication**(Isend and Irecv).
- Ensure that the receiving process performs a computation before printing the received value.
- How does Isend differ from send, and why is Wait() necessary in non-blocking communication?

Program



```
1 from mpi4py import MPI
2 import time
3
4 comm = MPI.COMM_WORLD
5 rank = comm.Get_rank()
6
7 if rank == 0:
8     data = 10
9
10    req = comm.isend(data, dest=1, tag=0)
11    req.wait()
12
13    print("Process 0 sent:", data)
14
15 elif rank == 1:
16    req = comm.irecv(source=0, tag=0)
17
18    print("Process 1 doing computation...")
19    time.sleep(2)
20
21    data = req.wait()
22
23    print("Process 1 received:", data)
```

Output

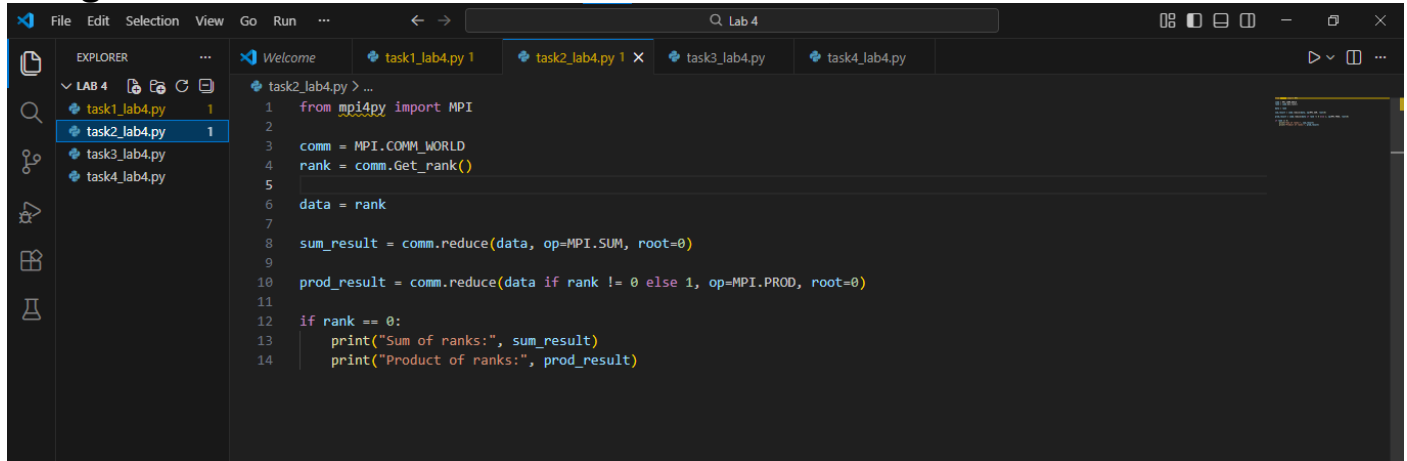
```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>mpiexec -n 2 python task1_lab4.py
Process 0 sent: 10
Process 1 doing computation...
Process 1 received: 10

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>
```

Task 2: Reduce Operation

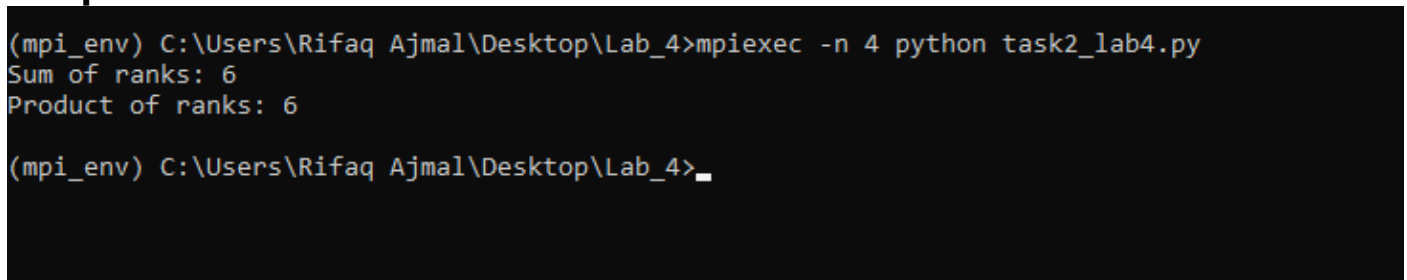
- Write an MPI program where each process initializes a variable with its **rank number** and uses the **Reduce operation** to compute the sum of all ranks at **Process 0**.
- Modify the program to use MPI.PROD instead of MPI.SUM. What difference do you observe in the result?

Program



```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 rank = comm.Get_rank()
5
6 data = rank
7
8 sum_result = comm.reduce(data, op=MPI.SUM, root=0)
9
10 prod_result = comm.reduce(data if rank != 0 else 1, op=MPI.PROD, root=0)
11
12 if rank == 0:
13     print("Sum of ranks:", sum_result)
14     print("Product of ranks:", prod_result)
```

Output



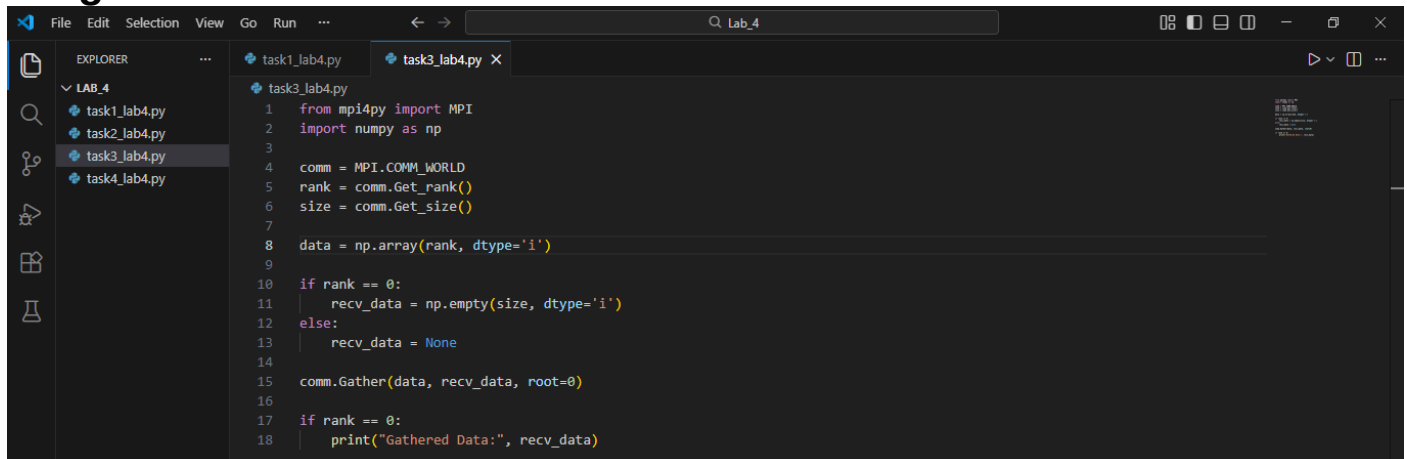
```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>mpiexec -n 4 python task2_lab4.py
Sum of ranks: 6
Product of ranks: 6

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>_
```

Task 3: Gather Operation (MPI.Gather)

- Implement an MPI program where each process initializes a number **equal to its rank** and sends it to **Process 0** using MPI.Gather.
- What happens if recv_data is not initialized correctly at the root process?
- Modify the program to use MPI.Gatherv for gathering different amounts of data from each process.

Program



```
1 from mpi4py import MPI
2 import numpy as np
3
4 comm = MPI.COMM_WORLD
5 rank = comm.Get_rank()
6 size = comm.Get_size()
7
8 data = np.array(rank, dtype='i')
9
10 if rank == 0:
11     recv_data = np.empty(size, dtype='i')
12 else:
13     recv_data = None
14
15 comm.Gather(data, recv_data, root=0)
16
17 if rank == 0:
18     print("Gathered Data:", recv_data)
```

Output

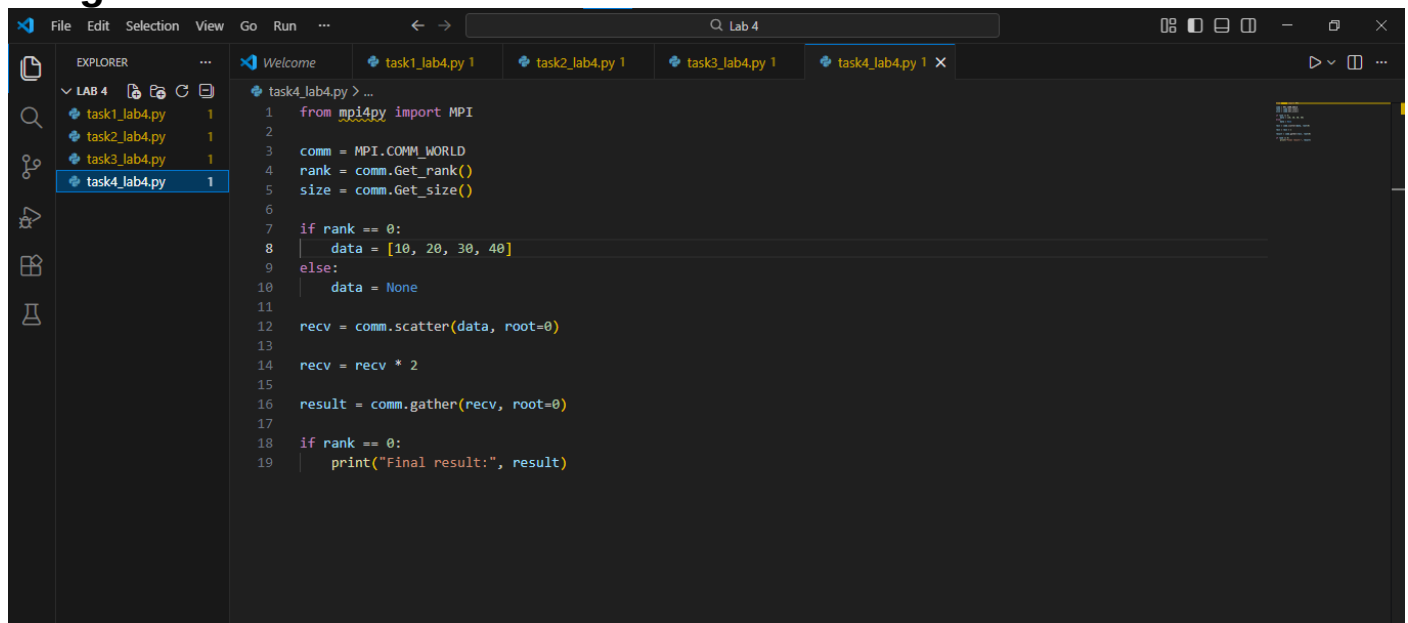
```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>mpiexec -n 4 python task3_lab4.py
Gathered Data: [0 1 2 3]

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>_
```

Task 4: Scatter Operation (MPI.Scatter)

- Write an MPI program where **Process 0** initializes an array [10, 20, 30, 40] (for 4 processes) and distributes one value to each process using MPI.Scatter.
- What happens if the number of processes is different from the number of elements in the array?
- Modify the program so that after receiving data, each process **doubles** its value and sends it back to **Process 0** using MPI.Gather.

Program



```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 rank = comm.Get_rank()
5 size = comm.Get_size()
6
7 if rank == 0:
8     data = [10, 20, 30, 40]
9 else:
10    data = None
11
12 recv = comm.scatter(data, root=0)
13
14 recv = recv * 2
15
16 result = comm.gather(recv, root=0)
17
18 if rank == 0:
19    print("Final result:", result)
```

Output

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>mpiexec -n 4 python task4_lab4.py
Final result: [20, 40, 60, 80]

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_4>
```